Why is my last test failing? I print something at the end of tearDown and it works:

```
class TestLoginServer(unittest.TestCase):
    @classmethod
    def setUpClass(cls) -> None:
        cls.login_port = 5000
        cls.lock = threading.Lock()
```

The most likely reason is a **port-reuse race across tests** combined with a **stale/partial `client` attribute**, but there are two concrete bugs that will bite you. Let me walk through what actually happens.

Bug 1: `test_03` is probably talking to a *leftover* server or a half-open connection

Look at `client_interaction`:

```python
self.client.sendall(msg)
response = self.client.recv(512)
if self.client.recv(512):   # <-- blocks until timeout or EOF
    self.fail()
self.client.close()
```

The second `recv(512)` is the problem. It returns:

- `b""` if the server closed the connection (good), **or**
- raises `socket.timeout` after 2 seconds if the server *hasn't* closed the connection yet, **or**
- returns data → `self.fail()`.

So every interaction that doesn't close the connection immediately costs you a full 2-second timeout. More importantly, if the server closed the socket but the FIN hasn't been processed, or if the server keeps the connection open, you get different behavior depending on timing. That's a flaky test, not a deterministic failure — which matches "my *last* test fails."

Bug 2: `tearDown` does not reset `self.client`, and `server_is_alive` is a red herring

In `tearDown`:

```python
if hasattr(self, "client") and self.client and self.client.fileno() != -1:
    self.client.close()
```

`self.client` is initialized in `setUp` **only as an annotation** (`self.client: socket.socket`) — that's a type hint, it does **not** create the attribute. So in `test_01` there is no `self.client` at all, which is fine because of `hasattr`. But in `test_02` / `test_03`, after `client_interaction` closes the socket, `self.client.fileno()` is `-1`, so `tearDown` skips closing again — fine. The real issue is elsewhere:

`tearDown` does `self.shutdown.set()` then `self.server.join(timeout=3)`. But `test_01` **never starts a server thread that reads `shutdown` in a way that exits**, and `test_02` / `test_03` start a fresh `self.server` each time on the **same port 5000**. If the previous `tearDown`'s `join` returned while the thread was still alive (because the login server doesn't check `self.shutdown` often enough or is blocked in `accept()`), the next test's `server.start()` will bind to port 5000 while an old listener still holds it → `client_interaction` connects to the **old** server, which has the **old** `self.stations` dict (from the previous test's `setUp`). That's why `test_03` fails even though `test_02` passed: `test_03`'s assertion checks `self.stations[2]` in the *new* dict, but the *old* server mutated the *old* dict.

This is exactly the kind of thing where `tearDown` "works" (you print and see it run, `assertFalse(self.server.is_alive())` passes because `join(3)` succeeded), yet the OS socket is still in `TIME_WAIT` / `LISTEN` from a zombie thread that didn't fully exit — or `SO_REUSEADDR` on the *client* (which does nothing for the listener) lets the new `bind()` succeed against a lingering listener.

Concrete fixes

1. **Don't use a fixed port.** Bind to port `0` in the server and read back the assigned port, or pick a fresh port per test from a helper. Fixed port 5000 across tests is the root cause of cross-test contamination.

2. **Make `tearDown` guarantee the listener is gone**, not just that the thread joined:

```python
def tearDown(self):
    self.shutdown.set()
    if getattr(self, "client", None):
        try:
            self.client.close()
        except OSError:
            pass
        self.client = None
    self.server.join(timeout=3)
    self.assertFalse(self.server.is_alive(), "server thread did not exit")
    # Wait until the port is actually free
    for _ in range(50):
        if not self.server_is_alive():
            break
        time.sleep(0.05)
    else:
        self.fail(f"port {self.login_port} still listening after teardown")
```

3. **Fix the double-`recv` in `client_interaction`.** If the protocol is "server closes after one message," just do one `recv` and rely on `b""`:

```python
response = self.client.recv(512)
if not response:
    self.fail("server closed without responding")
# optionally loop until b"" if multi-chunk is possible
```

If the server is *not* supposed to close, then the second `recv` is just a 2-second sleep and you should drop it or use `recv(..., MSG_DONTWAIT)` / a `select` with a short timeout.

4. `test_01` **doesn't actually test anything** — it starts the server and `tearDown` sets `shutdown`. If `login_server` is blocked in `accept()` and doesn't wake on the event, `join(3)` times out silently and you leak a listener. Check how `login_server` handles `shutdown` — it needs `socket.settimeout()` on the listener and a loop that checks `shutdown.is_set()`, otherwise `join` will always hit the timeout and `tearDown` will still "print at the end."

Why the print "works" but the test fails

Your `print` in `tearDown` runs *after* `self.server.join(timeout=3)` and the assertion, so it executes even when the join timed out and the assertion... wait — `assertFalse(self.server.is_alive())` inside a `try/except TimeoutError` will raise `AssertionError`, not `TimeoutError`, so it would fail the test *in* `tearDown`. If you're seeing the print, the thread *did* die. That means the failure is almost certainly **the stale-dict / wrong-server connection in** `test_03`, i.e., `client_interaction` connected to a listener that isn't the one whose `self.stations` you're asserting on — which is the fixed-port-5000 cross-test contamination above.

Short answer: pin down the port per test (or use port 0), and make `tearDown` wait until `server_is_alive()` is actually false before returning. The `test_03` assertion is checking the wrong dict because it connected to a leftover listener from `test_02`.